

OBJECT ORIENTED ANALYSIS AND DESIGN

CSA1103

OBJECT-ORIENTED ANALYSIS AND UML-BASED DESIGN OF AN AI-ENABLED SMART CAMPUS MANAGEMENT SYSTEM

ASSIGNMENT REPORT

Academic Details	Information
Course Name	Object Oriented Analysis and Design
Course Code	CSA1103
Assignment Weightage	100%
Course Outcomes Covered	CO1, CO2, CO3
Bloom's Taxonomy	BL3 – Apply BL4 – Analyse BL5 – Evaluate BL6 – Create
SDG Mapping	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
Student Name	P. Ranjithkumar
Register Number	192411119
Department / Section	Computer Science and Engineering (CSE)
Academic Year	2026–2027
Faculty Name	Dr.S.Ashok kumar

STUDENT DECLARATION / CODE OF CONDUCT

I, P. Ranjithkumar (192411119), certify that this assignment is my original academic work prepared for CSA11 – Object Oriented Analysis and Design. The analysis, object identification, UML models, design explanations and evaluation presented in this report have been prepared for the given AISCAMS case study. I have followed the assessment instructions and have used references where required. I understand that academic misconduct, plagiarism, fabrication or unauthorized copying may lead to disciplinary action.

Student Signature: P. Ranjithkumar

TABLE OF CONTENTS

1. Executive Summary
 2. Problem Analysis and Requirement Summary
 3. Actor Identification
 4. Use Case Identification and Descriptions
 5. Use Case Diagram
 6. Object and Class Identification
 7. Class Diagram
 8. Sequence / Interaction Diagrams
 9. Activity Diagram
 10. State Transition Diagram
 11. Package Diagram
 12. Component Diagram
 13. Deployment Diagram
 14. Object Responsibility Analysis
 15. Design Principles, Axioms and Patterns
 16. Assumptions and Constraints
 17. Requirement-to-Use Case/Class Traceability
 18. Implementation / Prototype and Results
 19. SDG Relevance
 20. Reflection and Learning Outcomes
 21. Conclusion
 22. GitHub Upload Structure
 23. References
- Appendix A – UML Artefact Checklist

LIST OF FIGURES

- Figure 5.1 – AISCaMS Use Case Diagram
- Figure 7.1 – AISCaMS Class Diagram
- Figure 8.1 – Sequence Diagram for AI-Assisted Classroom Booking
- Figure 8.2 – Sequence Diagram for Maintenance Request
- Figure 9.1 – Activity Diagram for Resource Recommendation and Booking
- Figure 10.1 – State Transition Diagram for Service Request
- Figure 11.1 – Package Diagram
- Figure 12.1 – Component Diagram
- Figure 13.1 – Deployment Diagram

1. EXECUTIVE SUMMARY

The AI-Enabled Smart Campus Management System (AISCAMS) is designed as a single digital platform for everyday campus activities. Instead of keeping classroom booking, academic schedules, maintenance complaints, events and facility information in separate places, the proposed system brings them together under one object-oriented design. Different users receive different permissions, so a student does not get the same controls as an administrator or maintenance employee.

A major part of the proposed system is the intelligent resource recommendation feature. When a user needs a classroom or facility, the system can consider time, capacity, equipment, location and previous usage information before presenting suitable options. The recommendation is only a suggestion; the final booking still passes through the normal availability and authorization checks.

The system also considers IoT-enabled campus infrastructure. Room occupancy sensors, environmental sensors and equipment devices can provide information to the platform. Maintenance staff can then work on service requests and update their progress. The UML models in this report are connected so that the use cases lead to objects and classes, while the sequence, activity and state models describe how those classes behave.

1.1 Project Vision

The vision is to create a practical campus management platform that is simple for users, organized internally, and flexible enough to add new services later. The design gives importance to clear responsibilities, high cohesion, low coupling, data protection and traceability between requirements and UML models.

1.2 Main Objectives

- Bring academic, facility and support services into one role-based system.
- Help students and faculty find and reserve suitable campus resources.
- Use an AI-based ranking approach to provide useful resource suggestions.
- Receive and display basic infrastructure information from IoT devices.
- Provide a clear service-request lifecycle for maintenance activities.
- Separate user-interface, application, domain and integration responsibilities.
- Apply object-oriented concepts such as encapsulation, inheritance and polymorphism.
- Produce consistent UML models that can be used as a blueprint for implementation.

2. PROBLEM ANALYSIS AND REQUIREMENT SUMMARY

2.1 Problem Domain

In a large campus, information is often handled by different departments. A student may use one method to check a timetable, another method to request a room, and a separate channel to report a damaged device. Faculty members may have additional academic requirements, while maintenance teams need a work list and status information. When these activities are disconnected, it becomes difficult to know which resource is free, who is responsible for a request, and whether an issue has already been reported.

AISCAMS addresses this problem through a common platform. The important design idea is not simply to store information, but to represent real campus entities as objects with clearly assigned responsibilities. A classroom becomes a Resource, a reservation becomes a Booking, a complaint becomes a ServiceRequest, and a repair assignment becomes a MaintenanceTask.

2.2 System Boundary

The system boundary includes authentication, academic information, resource search and booking, AI recommendation, service requests, maintenance tracking, event management, notifications, IoT monitoring and administrative reporting. External systems such as an institutional identity provider, IoT gateway and notification provider are treated as actors or connected services rather than internal domain classes.

2.3 Functional Requirements

ID	Functional Requirement
FR01	Authenticate users and apply role-based authorization.
FR02	Allow students and faculty to view relevant academic schedules.

FR03	Search classrooms and facilities using time, capacity, equipment and location.
FR04	Recommend suitable resources using availability and usage information.
FR05	Allow faculty to manage classes and attendance.
FR06	Allow authorized users to raise campus service requests.
FR07	Allow maintenance staff to accept, update and close assigned requests.
FR08	Allow administrators to manage users, resources, schedules, events and policies.
FR09	Collect occupancy, environmental and equipment status from IoT sources.
FR10	Allow authorized users to create and manage campus events.
FR11	Send notifications for important booking and service events.
FR12	Generate operational reports for authorized administrators.

2.4 Non-Functional Requirements

ID	Quality Attribute	Design Interpretation
NFR01	Security	Role-based access, secure sessions, least privilege and audit logging.
NFR02	Performance	Normal search and booking requests should complete within an acceptable campus-service response time.
NFR03	Scalability	The design should allow more users, buildings, resources and devices.
NFR04	Availability	Core campus services should remain available during normal operating periods.
NFR05	Maintainability	Modules and interfaces should allow changes without affecting unrelated areas.
NFR06	Usability	Screens should be understandable for students, faculty and staff.
NFR07	Reliability	Booking and service-request states should remain consistent.
NFR08	Interoperability	APIs and adapters should support external systems and IoT devices.
NFR09	Privacy	Personal, academic and operational information should be restricted to permitted roles.
NFR10	Observability	Logs and status records should

		support troubleshooting and accountability.
--	--	---

2.5 Requirements Prioritization

Priority	Requirements	Reason
Must Have	FR01, FR03, FR06, FR07, FR08, FR09	These are required for basic campus operation.
Should Have	FR04, FR05, FR11, FR12	They improve usefulness, intelligence and management visibility.
Could Have	FR02, FR10	They can be extended depending on the first implementation scope.

3. ACTOR IDENTIFICATION

Actor	Main Responsibility
Student	Checks schedule, searches resources, requests bookings, registers for events, raises service requests and receives updates.
Faculty	Manages classes and attendance, requests suitable rooms and raises academic or facility-related requests.
Administrator	Controls users, resources, schedules, policies, approvals, events and reports.
Security Personnel	Views authorized alerts and infrastructure information related to campus safety.
Maintenance Staff	Receives assigned jobs, records repair work, changes request status and closes completed work.
AI Recommendation Engine	Processes resource and usage information and returns ranked suggestions.
IoT Gateway	Transfers sensor and device readings into the campus platform.
Notification Gateway	Delivers email, push, SMS or in-app notifications.
Identity Provider	Validates institutional identity information during authentication.

3.1 Actor Relationships

Student, Faculty, Administrator, Security Personnel and Maintenance Staff are human actors with authenticated access. They share the general idea of a User, but their permissions are different. The AI engine, IoT gateway, notification gateway and identity provider are technical actors. Treating these integrations as external actors keeps the core domain model independent from a particular vendor or protocol.

4. USE CASE IDENTIFICATION AND DESCRIPTIONS

ID	Use Case	Actors	Purpose
UC01	Authenticate User	Student, Faculty, Administrator, Security, Maintenance	Check identity and create an authorized session.
UC02	View Academic Schedule	Student, Faculty	Show current and upcoming academic

			activities.
UC03	Search Campus Resource	Student, Faculty, Administrator	Find rooms or facilities using selected criteria.
UC04	Get AI Recommendation	Student, Faculty, Administrator	Rank suitable resources based on constraints and available data.
UC05	Book Classroom / Facility	Student, Faculty	Reserve a resource after final availability validation.
UC06	Manage Attendance	Faculty	Record and review attendance for assigned classes.
UC07	Raise Service Request	Student, Faculty, Administrator	Create a support request with category, location and description.
UC08	Process Maintenance Request	Maintenance Staff	Accept, work on, update and complete a maintenance request.
UC09	Monitor IoT Infrastructure	Administrator, Security, Maintenance	View device information, readings and important alerts.
UC10	Manage Events	Administrator, Faculty	Create, update, publish and manage campus events.
UC11	Send Notification	Notification Gateway	Deliver booking, request and system messages.
UC12	Generate Reports	Administrator	Create reports for utilization, maintenance and other operations.

4.1 Detailed Use Case – AI-Assisted Classroom Booking

Element	Description
Use Case	UC04 – Get AI Recommendation followed by UC05 – Book Classroom / Facility
Primary Actor	Student or Faculty
Precondition	The user is logged in and provides a date, time and resource requirements.
Main Flow	User enters requirements → system validates them → availability is checked → AI ranks possible resources → choices are displayed → user selects a resource → final availability check is performed → booking is stored → confirmation is sent.
Alternative Flow	If no suitable room is found, the system suggests another time, location or a less

	restrictive set of requirements.
Postcondition	A valid booking exists, or the user receives a meaningful alternative.
Business Rules	A confirmed booking cannot overlap another confirmed booking; restricted resources require permission; cancellation follows campus policy.

4.2 Detailed Use Case – Maintenance Request

Element	Description
Use Case	UC07 – Raise Service Request and UC08 – Process Maintenance Request
Primary Actor	Student / Faculty / Administrator / Maintenance Staff
Main Flow	Requester enters category, location and description → system creates the request → assignment service selects staff → staff accepts the work → progress is recorded → request is completed → requester receives an update.
Exception	If the category or location is invalid, the system asks for correction. If an active duplicate is found, the new request can be linked to the existing request.
Postcondition	The request is closed with a recorded outcome and responsible staff member.

5. USE CASE DIAGRAM

The use case model defines what users expect from AISCaMS. The system boundary contains the major services, while external technical actors are kept outside the boundary. The <<include>> relationships are used for activities that form a required part of another use case, such as checking availability before a booking.

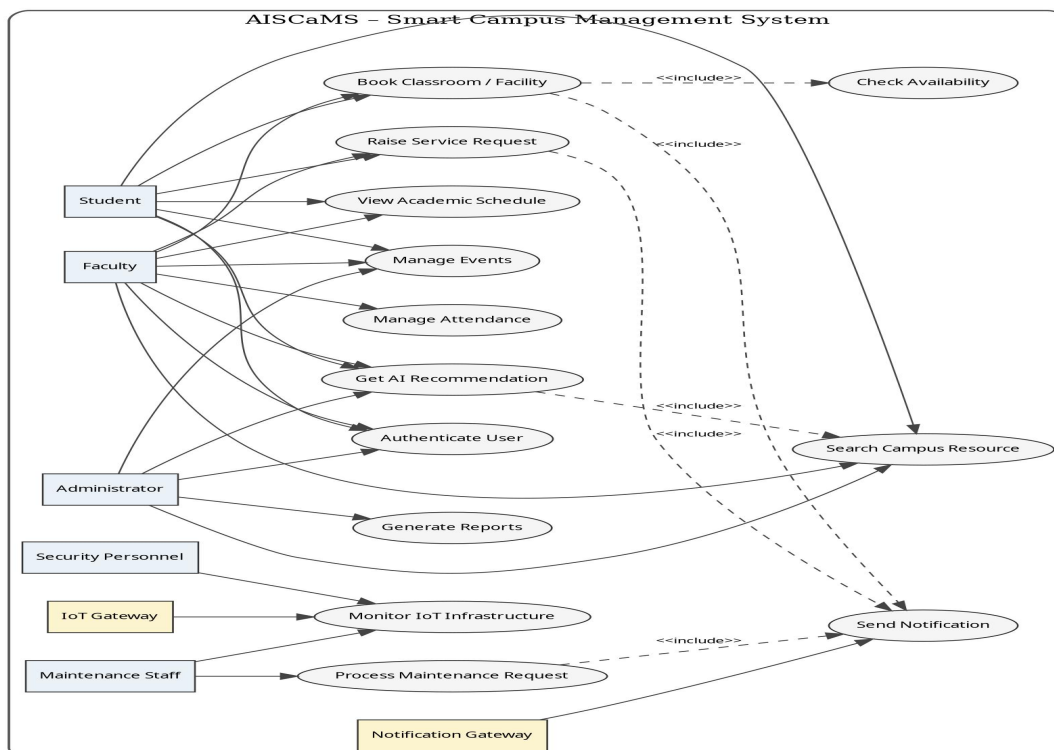


Figure 5.1 – AISCaMS Use Case Diagram.

The diagram shows that students have access to the largest set of everyday services, while administrators have wider management authority. Maintenance staff concentrate on service requests and infrastructure monitoring. The technical actors interact through defined interfaces instead of being treated as ordinary campus users.

6. OBJECT AND CLASS IDENTIFICATION

The object analysis starts by identifying important nouns in the problem statement and then checking whether each noun represents a real entity, a value, or a service responsibility. For example, Resource and Booking are domain entities, while RecommendationEngine is a service object. This separation prevents one class from becoming responsible for unrelated operations.

Class	Representative Attributes	Representative Methods
User	userId, name, email, role, status	authenticate(), authorize(), updateProfile()
Student	studentId, programme, semester	viewSchedule(), requestBooking(), raiseRequest()
Faculty	facultyId, department	manageClass(), recordAttendance(), requestFacility()
Administrator	adminId, privilegeLevel	manageUser(), manageResource(), generateReport()
ClassSession	sessionId, subject, startTime, endTime	assignRoom(), updateSchedule()
Resource	resourceId, name, type, capacity, status	isAvailable(), reserve(), release()
Booking	bookingId, startTime, endTime, status	confirm(), cancel(), validateOverlap()
ServiceRequest	requestId, category, priority, description, status	submit(), assign(), updateStatus(), close()
MaintenanceTask	taskId, assignedAt, completedAt, notes	start(), updateProgress(), complete()
IoTDevice	deviceId, type, location, status	publishTelemetry(), selfCheck()
Telemetry	telemetryId, timestamp, metric, value	validate(), normalize()
RecommendationEngine	modelVersion, policy	recommend(), rankResources(), explainScore()
Recommendation	recommendationId, score, reason	rank(), explain()
Notification	notificationId, channel, message, status	send(), markDelivered()
Event	eventId, title, venue, startTime, capacity	publish(), register(), cancel()
Report	reportId, type, generatedAt	generate(), export()

6.1 Relationship Analysis

Relationship	Example	Reason
Generalization	User → Student, Faculty, Administrator	Shared identity behaviour is kept in the parent class while role-specific operations remain

8.2 Maintenance Service Request

The maintenance sequence separates the person reporting an issue from the employee doing the repair. The request service creates the request and the assignment service finds an appropriate staff member. Notifications are handled by a separate service so that request processing does not depend on the internal details of an email or push provider.

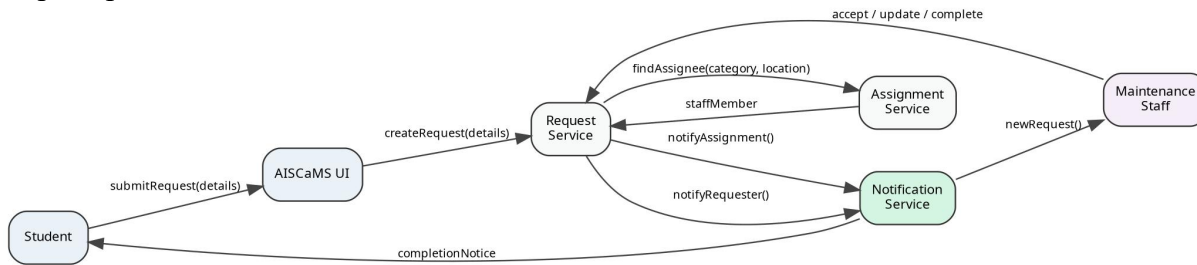


Figure 8.2 – Sequence Diagram for Maintenance Request.

9. ACTIVITY DIAGRAM

The activity model describes the complete path for a resource booking. It includes validation, resource search, AI ranking, user selection and a final availability decision. The two decision points are important because an invalid request should not enter the recommendation stage, and a resource that became unavailable should not be booked.

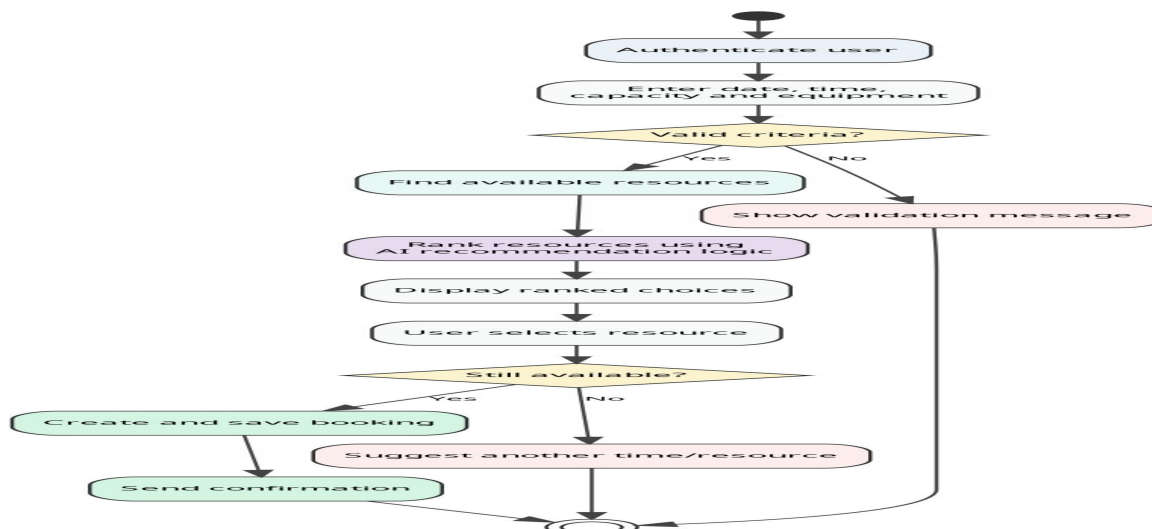


Figure 9.1 – Activity Diagram for Resource Recommendation and Booking.

10. STATE TRANSITION DIAGRAM

A service request has a clear lifecycle. It is first Submitted, then Assigned to a maintenance employee, Accepted, and moved to In Progress. If a required part or external dependency is unavailable, the request can move to On Hold. After the work is completed it becomes Resolved and finally Closed after verification.

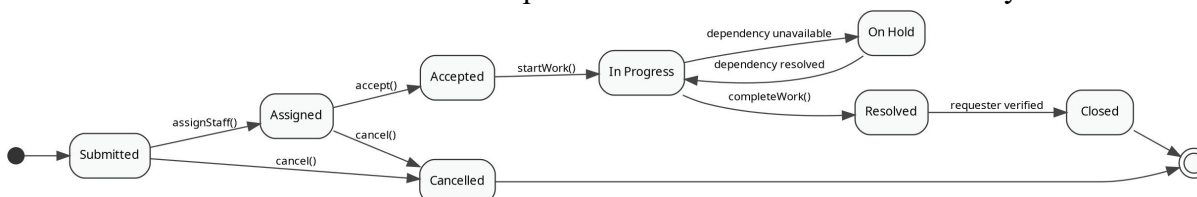


Figure 10.1 – State Transition Diagram for Service Request.

The state model prevents contradictory status values. For example, a request that is already Closed should not directly return to In Progress unless the system defines a separate reopen operation. This makes the lifecycle easier to audit and test.

11. PACKAGE DIAGRAM

The package model groups related classes and services according to responsibility. The Presentation package deals with user interaction, while domain-oriented packages handle academic, booking, maintenance and event activities. The AI and IoT areas are separated because they have different data and integration requirements.

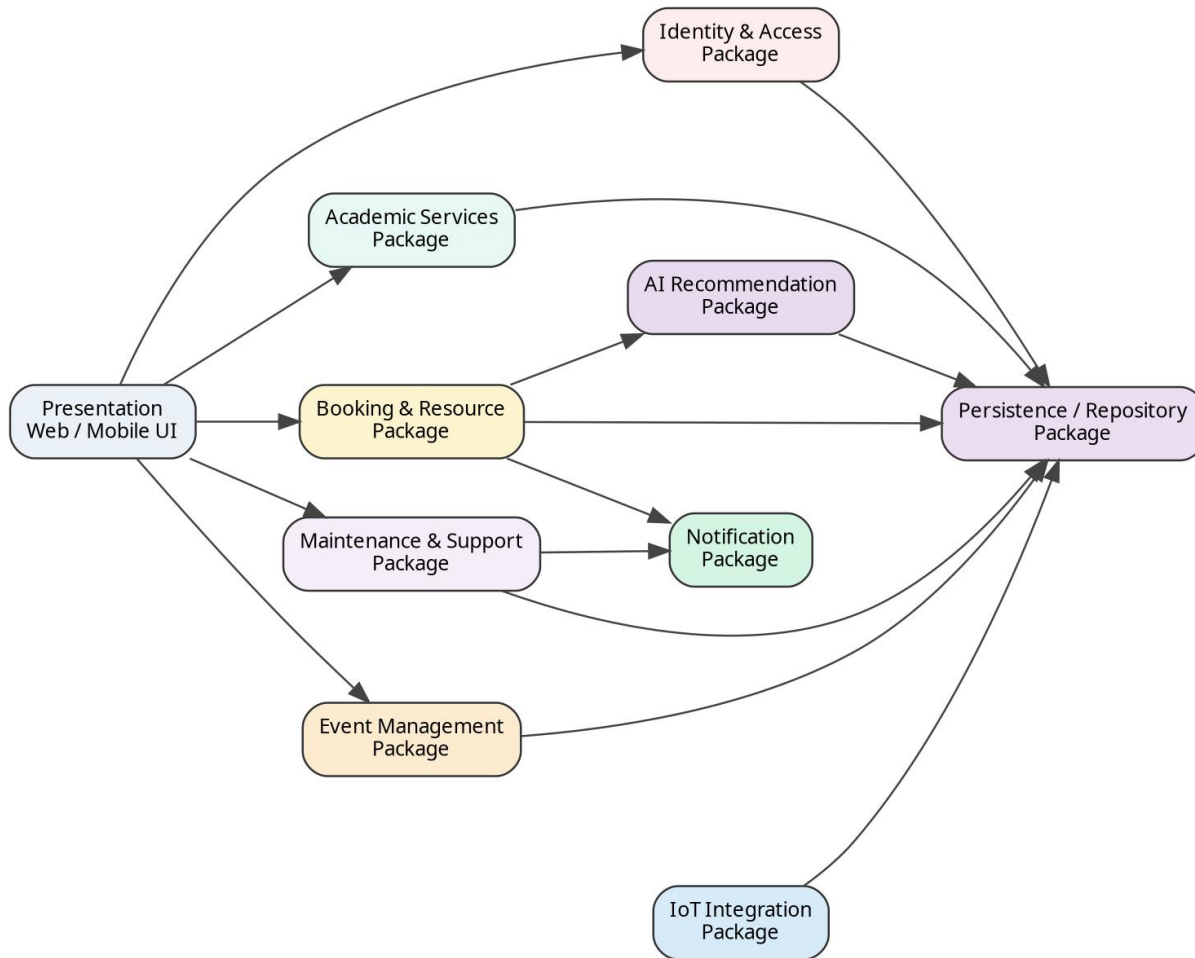


Figure 11.1 – AISCams Package Diagram.

The package arrangement supports high cohesion because related responsibilities stay together. It also supports low coupling because packages communicate through service interfaces instead of accessing each other's internal implementation.

12. COMPONENT DIAGRAM

The component model describes the software parts that could be implemented separately. The Web/Mobile Client communicates with an API Gateway. Behind the gateway, application services provide identity, academic, booking, maintenance and event functions. The recommendation service, IoT integration and notification service remain separate components.

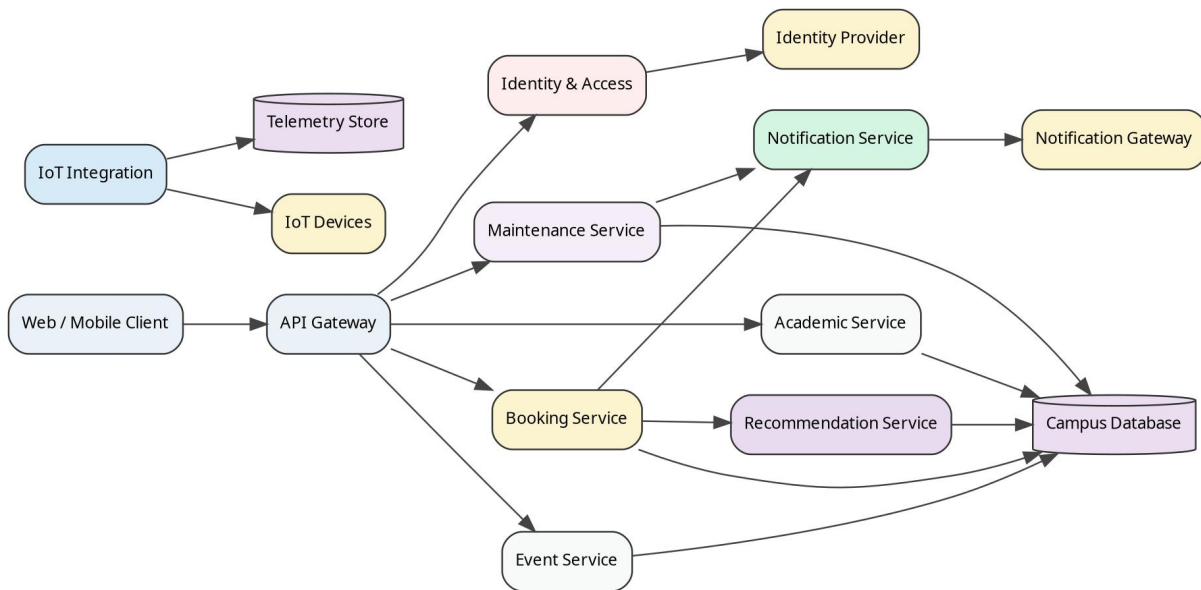


Figure 12.1 – AISCAMS Component Diagram.
 The database stores transactional information such as users, resources, bookings and requests. Telemetry can be stored separately because sensor data has a different access pattern and may grow much faster than normal campus records.

13. DEPLOYMENT DIAGRAM

The deployment model gives a logical view of where the software can run. Users access the system from a browser or mobile device. Requests pass through the campus network and API gateway before reaching an application cluster. The application connects securely to the campus database, telemetry store, identity provider and notification provider.

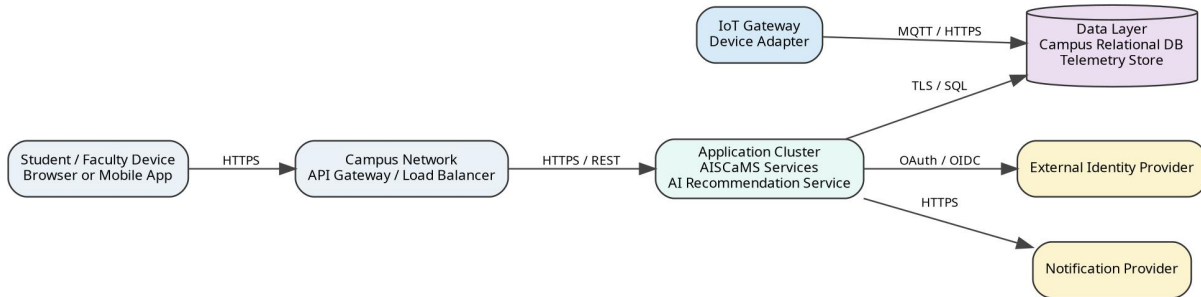


Figure 13.1 – AISCAMS Deployment Diagram.
 The use of a gateway or load balancer makes it possible to add application instances when the number of users increases. External identity and notification services are kept outside the application cluster so that the main system does not depend on a single internal implementation.

14. OBJECT RESPONSIBILITY ANALYSIS

Object / Class	Assigned Responsibility	Design Rationale
User	Own common identity and authorization behaviour.	Keeps common behaviour in one place and avoids duplication.
Student	Perform student-specific service actions.	Prevents student workflow from being placed in the generic User class.
Faculty	Manage teaching activities and academic requirements.	Keeps faculty responsibilities separate from student functions.
Resource	Maintain resource availability and reservation state.	The resource is the information expert for its own availability.

Booking	Represent a reservation and enforce booking rules.	Booking owns the state and overlap invariant of the transaction.
RecommendationEngine	Rank possible resources.	Keeps AI logic independent of booking persistence.
ServiceRequest	Maintain request category, priority and lifecycle.	The request controls its own valid status changes.
MaintenanceTask	Track actual repair work.	Separates the request record from the work performed by staff.
IoTDevice	Publish telemetry and device-health information.	Hides device-specific integration details.
Notification	Represent delivery status.	Allows different channels without changing business entities.

14.1 GRASP-Oriented Responsibility Assignment

Principle	AISCaMS Application
Information Expert	Resource checks availability because it owns resource state; Booking controls reservation state.
Creator	Booking Service creates a Booking because it coordinates the reservation data and validation.
Controller	Application services coordinate use cases rather than placing orchestration inside the UI.
Low Coupling	Services use interfaces and repositories instead of depending directly on vendor-specific infrastructure.
High Cohesion	Recommendation, booking, maintenance and notification responsibilities are separated.
Polymorphism	Different notification channels and recommendation strategies can follow common interfaces.
Protected Variations	Adapters isolate changes in identity, IoT and notification providers.

15. DESIGN PRINCIPLES, AXIOMS AND PATTERNS

15.1 Object-Oriented Design Principles

Principle	Application in AISCaMS
Encapsulation	Booking and service-request state is changed through controlled operations rather than unrestricted fields.
Abstraction	Interfaces hide repository, identity, notification and IoT implementation details.
Inheritance	Student, Faculty and Administrator reuse common User behaviour.
Polymorphism	Notification and recommendation strategies can be exchanged behind common interfaces.
High Cohesion	Each class or service has a focused responsibility.

Low Coupling	Domain logic is kept independent from external APIs and database technology.
Single Responsibility	A class or service has one main reason to change.
Open / Closed	New notification channels or ranking strategies can be added without rewriting booking rules.
Dependency Inversion	Application logic depends on abstractions rather than concrete infrastructure.

15.2 Design Axioms

Axiom 1 – Independence of Components: Each module should have limited dependence on other modules. A change in the notification provider, for example, should not require changes to Booking.

Axiom 2 – Minimize Complexity: The design divides the problem into domain entities, application services and adapters. This reduces the amount of responsibility carried by individual classes and makes testing easier.

15.3 Design Pattern Justification

Pattern	Reason for Selection
Strategy	Useful when resource ranking or notification delivery may use different algorithms or policies.
Observer	Suitable for booking, maintenance or infrastructure events that need to inform several listeners.
Factory	A NotificationFactory can create the required notification channel from configuration.
Adapter	Useful for converting different IoT, identity or notification APIs into common AISCAMS interfaces.
Repository	Keeps database operations separate from domain and application logic.
Facade	Provides a simple entry point when a user action needs several internal services.

16. ASSUMPTIONS AND CONSTRAINTS

16.1 Assumptions

- Users are provided with valid institutional identities and roles.
- Campus resources contain basic information such as capacity, equipment, location and status.
- Campus scheduling uses a consistent time zone and booking policy.
- The AI recommendation feature is advisory and cannot override authorization rules.
- IoT devices may sometimes send delayed or incomplete readings.
- Administrators define which users may access restricted rooms and facilities.
- Notification delivery depends on the availability of the external notification provider.

16.2 Constraints

- All UML diagrams must describe the same system boundary and vocabulary.
- Operations in the class diagram should be reflected by relevant sequence or activity interactions.
- Confirmed bookings must not overlap for the same resource and time period.
- Sensitive data must only be visible to authorized roles.
- AI recommendations must pass through final business-rule validation before any booking is stored.
- The architecture should allow additional campus services without redesigning the entire system.

17. REQUIREMENT-TO-USE CASE/CLASS TRACEABILITY

Requirement	Use Case	Primary Classes	Module
FR01	UC01 Authenticate User	User, Student, Faculty, Administrator	Identity & Access
FR02	UC02 View Academic Schedule	Student, Faculty, ClassSession	Academic Service
FR03	UC03 Search Resource; UC05 Book Resource	Resource, Booking	Booking & Resource
FR04	UC04 Get AI Recommendation	RecommendationEngine, Recommendation, Resource	Recommendation Service
FR05	UC06 Manage Attendance	Faculty, ClassSession	Academic Service
FR06	UC07 Raise Service Request	ServiceRequest, User	Maintenance Service
FR07	UC08 Process Maintenance Request	MaintenanceTask, ServiceRequest	Maintenance Service
FR08	UC03, UC10, UC12	Administrator, Resource, Event, Report	Administration
FR09	UC09 Monitor IoT Infrastructure	IoTDevice, Telemetry	IoT Integration
FR10	UC10 Manage Events	Event, User	Event Service
FR11	UC11 Send Notification	Notification	Notification Service
FR12	UC12 Generate Reports	Report, Administrator	Reporting

Traceability is important because it shows that requirements are not isolated statements. For example, FR03 is represented through resource search and booking use cases, which use the Resource and Booking classes and are implemented through the booking/resource module. The same method can be followed for every functional requirement.

18. IMPLEMENTATION / PROTOTYPE AND RESULTS

18.1 Prototype Architecture

A prototype can be developed as a web application with a responsive client, REST APIs, a relational database and a separate recommendation service. The initial implementation does not need every smart-campus feature at once. A practical first version can concentrate on login, resource search, booking, service requests and basic administration, followed by AI ranking and IoT integration.

18.2 Suggested Technology Stack

Layer	Possible Technology	Purpose
Client	React / Next.js	Role-based web interface for campus users.
Backend	Python FastAPI / Node.js / Java Spring Boot	REST APIs and application services.
Database	MySQL / PostgreSQL	Users, resources, bookings, requests and events.
AI Module	Python recommendation service	Ranking resources from constraints and usage data.
IoT / Messaging	MQTT / message broker	Receiving device and sensor events.

Authentication	OAuth 2.0 / OpenID Connect	Institutional identity integration.
Deployment	Docker and a cloud/container platform	Repeatable deployment and scaling.

18.3 Prototype Workflow

1. User logs in and the system identifies the role.
2. Student or faculty member enters the required date, time, capacity and equipment.
3. The booking service requests available resources.
4. The recommendation service ranks suitable options.
5. The user selects one option and the booking service checks availability again.
6. The booking is stored only after the final validation succeeds.
7. A notification is sent to the user.
8. For a support issue, the user creates a service request and the assignment service routes it to maintenance staff.

18.4 Expected Results

Area	Expected Result
Authentication	Users can enter only the functions permitted for their role.
Resource Management	Authorized users can search and reserve available resources.
AI Recommendation	Suitable rooms or facilities are displayed in ranked order.
Maintenance	Requests have visible status and responsible staff.
IoT Monitoring	Recent device readings can be displayed for authorized users.
Notifications	Users receive important status updates.
Architecture	Separate modules make later changes easier to manage.

18.5 Testing Plan

Test Area	Test Condition	Expected Result
Authentication	Invalid password	Access is rejected and no authorized session is created.
Authorization	Student attempts an admin operation	Operation is denied.
Booking	Two users request the same room and time	Only one confirmed booking is accepted.
Recommendation	No resource matches all conditions	System proposes alternatives.
Maintenance	Staff changes Accepted to In Progress	Status changes successfully and is recorded.
IoT	Malformed telemetry is received	Invalid reading is rejected or quarantined.
Notification	External gateway is unavailable	Failure is recorded and retry handling is applied.
Reporting	Administrator requests a utilization report	Report is produced from authorized records.

19. SDG RELEVANCE

SDG	Relevance to AISCaMS
SDG 4 – Quality Education	Better access to academic schedules, learning spaces, classrooms and support services can make campus resources easier to use.
SDG 9 – Industry, Innovation and Infrastructure	The system combines software services, AI, APIs and IoT data to support a modern digital campus infrastructure.
SDG 11 – Sustainable Cities and Communities	Occupancy information, resource utilization and maintenance tracking can reduce unnecessary use of facilities and support efficient campus operations.

The SDG mapping is connected to the actual design rather than being treated as a separate topic. For example, smart occupancy information can help identify under-used spaces, while maintenance tracking can improve the life cycle of campus equipment.

20. REFLECTION AND LEARNING OUTCOMES

The main learning from this assignment is that object-oriented analysis begins with understanding the problem, not with drawing a class diagram immediately. In AISCaMS, many entities appear related at first, but their responsibilities are different. A Booking should not manage IoT readings, and a RecommendationEngine should not directly change a booking state.

Another important point is consistency between diagrams. The same Resource and Booking concepts are used in the use case, class, sequence and activity models. The service-request state diagram also gives a more detailed view of the lifecycle represented by the maintenance use case. This helped in understanding why UML diagrams should be developed as connected models rather than independent pictures.

20.1 Learning Outcome Mapping

Course Outcome	Evidence in the Assignment
CO1	Problem analysis, requirements, object identification, development approach and OO system design decisions.
CO2	Use Case, Class, Sequence, Activity, State Transition, Package, Component and Deployment diagrams.
CO3	Objects, classes, relationships, responsibilities, design principles, patterns, traceability, cohesion and coupling.

The assignment also helped in understanding the practical value of GRASP responsibility assignment and common design patterns. Instead of adding patterns only for decoration, the patterns were linked to actual changes that may occur, such as adding another notification channel or replacing an external IoT provider.

21. CONCLUSION

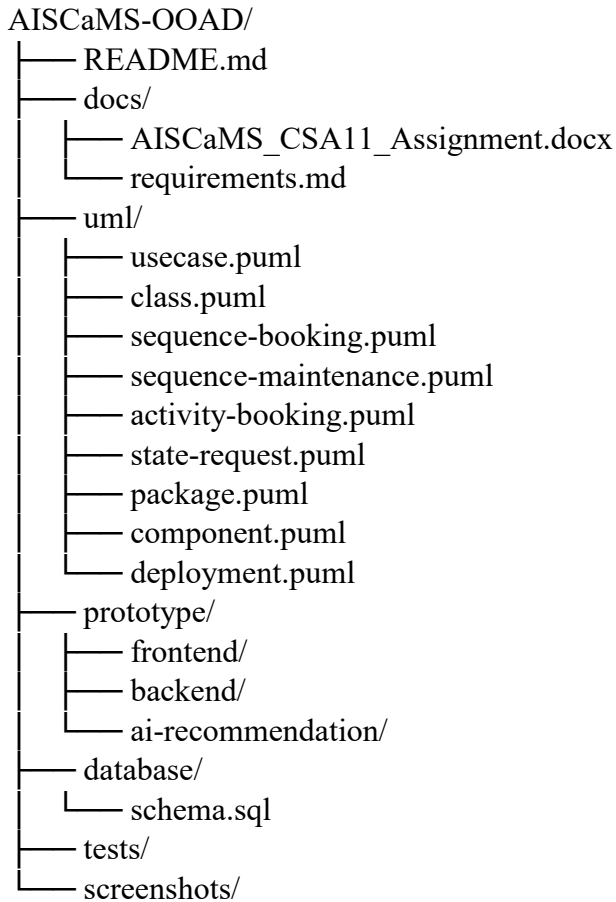
AISCaMS provides a structured object-oriented blueprint for a smart campus platform. The proposed design starts from the campus problem, identifies actors and use cases, converts important domain concepts into classes, and then extends the analysis into behavioural and architectural UML models.

The design emphasizes clear responsibility assignment, encapsulation, low coupling and high cohesion. The AI recommendation feature is separated from the booking transaction, while IoT and notification integrations are isolated through adapters and interfaces. This approach makes the system easier to maintain and provides a reasonable foundation for future implementation.

Overall, the assignment demonstrates how OOAD can be used to move from a real-world problem statement to a consistent software design that can later be implemented, tested and extended.

22. GITHUB UPLOAD STRUCTURE

The final repository can be organized so that the UML models, report and prototype files are easy to locate. A suggested structure is shown below.



22.1 GitHub Upload Checklist

- Add the final report to the docs folder.
- Upload all UML source files, not only exported images.
- Include a README explaining the project, actors and major modules.
- Keep prototype code separated into frontend, backend and AI folders.
- Add database schema and test files if they are part of the prototype.
- Do not upload passwords, API keys or private credentials.

23. REFERENCES

1. Object Management Group (OMG), Unified Modeling Language (UML) specification and modelling concepts.
2. Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J., and Houston, K. A., Object-Oriented Analysis and Design with Applications.
3. Larman, C., Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Iterative Development.
4. Gamma, E., Helm, R., Johnson, R., and Vlissides, J., Design Patterns: Elements of Reusable Object-Oriented Software.
5. Sommerville, I., Software Engineering.
6. Martin, R. C., Agile Software Development, Principles, Patterns, and Practices.
7. Hadzhikoleva, S., Hadzhikolev, E., Gaftandzhieva, S., and Pashev, G. (2025), A conceptual framework for multi-component summative assessment in an e-learning management system, *Frontiers in Education*.
8. Luo, J., Zheng, C., Yin, J., et al. (2025), Design and assessment of AI-based learning tools in higher education: A systematic review, *International Journal of Educational Technology in Higher Education*.

9. Banihashem, S. K. (2025), A critical review of using learning analytics for formative assessment: Progress, pitfalls and path forward, Journal of Computer Assisted Learning.
10. Üstün, A. G., Kayalı, B., and Yavuz, M. (2026), Artificial Intelligence Supported Online Assessment and Evaluation in Higher Education: A Systematic Review With SWOT Analysis, SAGE Open.
11. Course lecture notes and laboratory materials for CSA11 – Object Oriented Analysis and Design.

APPENDIX A – UML ARTEFACT CHECKLIST

No.	Deliverable	Included	Consistency Check
1	Problem Analysis and Requirement Summary	Yes	Checked against the requirement and related UML models.
2	Actor Identification	Yes	Checked against the requirement and related UML models.
3	Use Case Descriptions	Yes	Checked against the requirement and related UML models.
4	Use Case Diagram	Yes	Checked against the requirement and related UML models.
5	Object and Class Identification	Yes	Checked against the requirement and related UML models.
6	Class Diagram	Yes	Checked against the requirement and related UML models.
7	Sequence / Interaction Diagram(s)	Yes	Checked against the requirement and related UML models.
8	Activity Diagram	Yes	Checked against the requirement and related UML models.
9	State Transition Diagram	Yes	Checked against the requirement and related UML models.
10	Package Diagram	Yes	Checked against the requirement and related UML models.
11	Component Diagram	Yes	Checked against the requirement and related UML models.
12	Deployment Diagram	Yes	Checked against the requirement and related UML models.
13	Object Responsibility Analysis	Yes	Checked against the requirement and related UML models.
14	Design Axioms / Principles and Design Pattern Justification	Yes	Checked against the requirement and related UML models.

15	Assumptions and Constraints	Yes	Checked against the requirement and related UML models.
16	Requirement-to-Use Case/Class Traceability	Yes	Checked against the requirement and related UML models.
17	Implementation / Prototype and Results	Yes	Checked against the requirement and related UML models.
18	Reflection on Design Decisions and Learning Outcomes	Yes	Checked against the requirement and related UML models.
19	GitHub Upload	Yes	Checked against the requirement and related UML models.

Term	Meaning in AISCams
Actor	A person, organization or external system that interacts with AISCams.
Object	A runtime or conceptual instance representing a campus entity or service.
Class	A definition of the attributes and operations shared by related objects.
Association	A relationship between two model elements that interact or know each other.
Generalization	An inheritance relationship in which a specialized class derives common features from a parent class.
Aggregation	A whole-part relationship where the part can exist independently.
Composition	A stronger whole-part relationship where the part belongs to the lifecycle of the whole.
Dependency	A relationship in which one element relies on another for some operation or information.
Encapsulation	Keeping internal state protected and controlling changes through operations.
Polymorphism	Allowing different implementations to be used through a common abstraction.